CAHIER · JOURS 1 ET 2

Context Engineering, skills & workflows

Du prompt improvisé au harness maîtrisé

5 Degrés · jeudi 15 et vendredi 16 octobre 2026 · en présentiel

Animateur : Anthony Denneulin

Version de référence : Claude Code 2.1.296 (9 octobre 2026). L'outil change chaque semaine : en cas de doute, la documentation officielle fait foi.

Mode d'emploi

1. Le livret a deux parties : ce cahier, qui sert pendant la formation, et le récap, à relire après, publié le vendredi 16 octobre à 16h30. Les deux sont sur la page du livret jusqu'au 23 octobre : denneulin.github.io/livret-context-engineering.
2. Enregistrez le fichier Markdown du cahier hors de votre repo (le dossier Téléchargements suffit) et remplissez-le dans votre éditeur : chaque « … » est une case à remplir. Dans votre repo, Claude le lirait comme le reste du code, et un `git add -A` l'embarquerait.
3. Le cahier ne s'ouvre qu'aux exercices, aux ateliers et en fin de demi-journée : pendant la théorie, il n'y a rien à suivre.
4. Les fichiers d'exercice (hooks, réglages, modèle de skill) et les commandes et prompts du J2 (installation, présentation du projet, chaîne de skills, conditions `/goal`, pipeline, analyse parallèle, serveur MCP) sont reproduits en entier : copiez-les plutôt que de les retaper.
5. Le soir du J1, vous choisissez votre ticket fil rouge (fiche à la fin du cahier du J1 après-midi) : un ticket de votre projet, ou, sans projet sous la main, un ticket de Tailspin Toys, le projet d'entraînement du J2, choisi sur place. Il traverse tout le J2 : la spec, les tickets et une première loop le matin, l'exécution en parallèle l'après-midi.
6. Le plan d'adoption à 30 jours se construit ligne par ligne, une ligne à la fin de chaque demi-journée ; vous le consolidez en clôture du J2, en un plan de quatre semaines.
7. Les listes à cocher des exercices aident à relire votre travail : les cocher est facultatif. La checklist context engineering, elle, se coche dans les semaines qui suivent.

Sommaire

Partie	Contenu
J1 matin — Cahier	Premier contact, réécrire un prompt, Atelier 1
J1 après-midi — Cahier	Exercice `.env`, exercice skill, ticket fil rouge
J2 matin — Cahier	Votre projet ou Tailspin Toys, le projet présenté par Claude, votre ticket du jour, Atelier 2 en quatre étapes et son plan de composition, « `/goal` à blanc », la condition d'arrêt
J2 après-midi — Cahier	La préparation des worktrees, Atelier 3 en cinq étapes et ses deux prompts, grille des tentations, serveur MCP
Plan d'adoption à 30 jours	Une ligne par demi-journée, puis la consolidation en quatre semaines
Checklist context engineering	35 pratiques à adopter : 15 du J1, 20 du J2
Auto-évaluation	Les 6 objectifs du programme, de 1 à 4

J1 matin — Cahier

Exercice : premier contact avec votre repo (10 min)

1. Ouvrez un terminal à la racine de votre repo et lancez `claude`.
2. Passez en Plan mode (Shift+Tab).
3. Demandez :

```
Explique l'architecture de ce projet et liste les commandes pour builder, tester et linter.
```

4. Notez ce qui est juste, faux ou manquant : ce sera votre matière pour l'Atelier 1.
5. Dans la même session, tapez `/context` : qu'est-ce qui prend le plus de place ?

Question	Vos notes
Repo et stack	
Ce que Claude a dit de l'architecture	
Les commandes qu'il a trouvées (build, test, lint)	
Ce qui est juste	
Ce qui est faux	
Ce qui manque	
`/context` : ce qui prend le plus de place	

Si ça coince : Claude Code n'est pas installé ou pas connecté → travaillez sur le poste de votre binôme. Repo très gros → lancez `claude` dans un sous-dossier. Projet .NET : si Claude ne trouve pas les commandes, ce sont souvent `dotnet build` et `dotnet test`.

Exercice express : réécrire un prompt (4 min)

Prenez un prompt flou que vous avez tapé cette semaine, ou celui-ci : « Optimise la page de recherche. » Réécrivez-le avec les quatre verbes.

Avant : le prompt flou	Après : avec les quatre verbes

Vérifiez votre version « après » :

- [ ] Elle référence un fichier ou un symbole (`@chemin`).
- [ ] Elle montre un modèle à suivre.
- [ ] Elle contraint : ce qu'il ne faut pas toucher, et un critère de vérification.
- [ ] Elle structure : un plan d'abord, des étapes ou un format de sortie.

Atelier 1 : auditer et améliorer votre CLAUDE.md (45 min, en binôme)

Phase	Durée	Ce que vous faites
1. Passer au crible	12 min	Ni CLAUDE.md ni AGENTS.md ? `/init` d'abord (un AGENTS.md seul se passe au crible tel quel : créer un CLAUDE.md le ferait ignorer). Pour chaque ligne : « si je la retire, Claude fera-t-il une erreur ? » Si non, coupez. Ajoutez ce que Claude a raté au premier contact.
2. Relecture croisée	10 min	Votre binôme relit votre CLAUDE.md avec la grille ci-dessous, et vous le sien.
3. Améliorer et extraire une règle	15 min	Appliquez au moins l'amélioration proposée par votre binôme, par exemple une règle rigide reformulée en principe. Extrayez une règle `.claude/rules/` avec `paths`, puis vérifiez qu'elle se charge.
4. Contre-vérifier	8 min	Lancez `/doctor prompt-audit` : qu'a-t-il trouvé que vous aviez raté ? Qu'est-ce que vous refusez dans ses propositions ?

Livrable : votre CLAUDE.md et au moins une règle, commités.

Restitution (2 minutes par binôme) : la section la plus utile de votre CLAUDE.md ; ce que l'audit a révélé par rapport à votre propre passe ; la règle que vous avez extraite.

Si ça coince : la conversation en cours garde l'ancienne version du CLAUDE.md → `/clear` pour tester la nouvelle. Règle qui ne se charge pas → vérifiez le glob (relatif à la racine du repo), faites lire à Claude un fichier qui correspond, et guettez la ligne « Loaded .claude/rules/… » dans le terminal. `/doctor prompt-audit` refuse de se lancer → vérifiez `claude --version` (2.1.283 ou plus) ; sinon, la passe manuelle était l'exercice.

Grille de relecture : le CLAUDE.md de votre binôme

Critère	Note	La question
Spécificité	1 · 2 · 3	Des commandes exactes ? Un nouveau pourrait-il s'en servir ?
Bruit	1 · 2 · 3	Chaque section apporte-t-elle quelque chose que le code ne dit pas ?
Boundaries	oui · non	Les zones interdites sont-elles listées ?
Taille	… lignes	Moins de 200 lignes ?

Chaque relecteur donne un point fort et une amélioration concrète.

Retour	Vos notes
Un point fort	
Une amélioration concrète	

Checklist du livrable

- [ ] Le CLAUDE.md fait moins de 200 lignes.
- [ ] Il donne les commandes exactes pour builder, tester et linter.
- [ ] Il signale les pièges et l'environnement (variables, services, options non évidentes), sans décrire le code fichier par fichier.
- [ ] Il désigne un fichier modèle (golden file) au lieu d'énumérer des règles de style.
- [ ] Il liste les zones interdites (Boundaries).
- [ ] Les erreurs relevées au premier contact sont corrigées.
- [ ] Au moins une règle `.claude/rules/*.md` avec `paths`, et vous avez vérifié qu'elle se charge.
- [ ] Le CLAUDE.md et la règle sont commités.

J1 après-midi — Cahier

Exercice : protéger votre `.env` (30 min)

Objectif : qu'aucun secret de votre projet n'entre dans la conversation. Vous superposez deux protections, une règle `deny` puis un hook, et vous cherchez ce qui passe encore.

Avant de commencer. Vous allez essayer de faire lire votre `.env` par Claude ; si un essai réussit, son contenu part dans la conversation. Travaillez donc avec un faux secret : déplacez votre vrai `.env` hors du repo le temps de l'exercice et créez un `.env` de test contenant `FAKE_SECRET=abc123`, ignoré par git. Remettez le vrai à la fin. Projet .NET : même précaution pour `appsettings.Development.json` si vous voulez tester ce fichier. Vérifiez aussi que Node.js répond : `node --version`.

Où mettre les fichiers :

```
mon-projet/
├── .claude/
│   ├── settings.json         ← la règle deny et le hook
│   └── hooks/
│       └── protect-env.mjs   ← le script
└── .env                      ← le faux secret de test
```

Si votre projet a déjà un `.claude/settings.json`, ne l'écrasez pas : ajoutez-y les clés `permissions.deny` et `hooks`.

Étape 1 : la règle `deny` (5 min)

Dans `.claude/settings.json` :

```
{
  "permissions": {
    "deny": [
      "Read(**/.env)",
      "Edit(**/.env)"
    ]
  }
}
```

Relancez Claude Code, puis demandez « Lis le fichier .env » : la lecture est refusée. Demandez ensuite « Affiche le .env avec cat » : dans la version testée (Claude Code 2.1.296), la règle bloque aussi `cat`, `head` ou `grep` lancés sur le fichier nommé. Essayez alors « Lis le .env avec un petit script node » : si ça passe, vous avez trouvé le trou que le hook va boucher. Autre limite : `Read(**/.env)` ne vise que les fichiers nommés exactement `.env`, pas `.env.local`.

Étape 2 : le hook `PreToolUse` (15 min)

1. Copiez `protect-env.mjs` (ci-dessous) dans `.claude/hooks/`.
2. Ajoutez à `.claude/settings.json` la clé `hooks` de `settings.exemple.json` (ci-dessous) : le hook est branché sur Read, Edit, Write, Bash, PowerShell et Grep. PowerShell et Grep pour Windows, où Claude Code peut lancer ses commandes avec l'outil PowerShell au lieu de Bash, et chercher avec l'outil Grep ; sur macOS et Linux, il cherche avec `grep` par Bash.
3. Testez le script seul, sans Claude. Le terminal doit afficher le message de blocage, puis `2`.

Bash ou zsh (macOS, Linux, Git Bash) :

```
echo '{"tool_input":{"command":"cat .env"}}' | node .claude/hooks/protect-env.mjs
echo $?
```

PowerShell :

```
'{"tool_input":{"command":"cat .env"}}' | node .claude/hooks/protect-env.mjs
$LASTEXITCODE
```

4. Relancez Claude Code et testez :

Prompt à taper dans Claude Code	Résultat attendu
« Affiche le contenu du .env avec cat »	Bloqué : Claude reçoit le message « Bloqué par le hook protect-env... »
« Lis le fichier .env »	Refusé (règle `deny` ou hook)
« Lis le .env.example », si vous en avez un	Autorisé : fichier d'exemple versionné
« Lis » suivi d'un fichier de code de votre projet	Autorisé

Si Claude refuse de lui-même avant que le hook intervienne, tant mieux, mais ce n'est pas une garantie : c'est son jugement, pas une barrière. Le test à la main (point 3) prouve que le hook bloque.

Projet .NET. `appsettings.Development.json`, `appsettings.Production.json` et `secrets.json` sont déjà dans la liste du script. Pour ajouter un fichier, complétez la liste `AUTRES_SECRETS` en minuscules, par exemple `"appsettings.staging.json"`.

Étape 3 : la chasse à la faille (5 min)

Essayez de faire lire le faux secret par Claude sans que le hook le voie : une autre commande, un autre chemin, un autre outil. Notez ce qui passe.

Ce que j'ai essayé	Bloqué ?	Pourquoi

Si vous séchez : un joker du shell, un nom de fichier fabriqué au moment de l'exécution, une recherche sur tout le dossier (sur Mac et Linux, avec `--no-ignore-files` : le grep de Claude Code saute les fichiers ignorés par git), un fichier de secrets qui ne s'appelle pas `.env`. Côté .NET : un outil qui affiche les secrets utilisateur sans jamais nommer `secrets.json`.

Ce que ça montre : le hook lit le texte de la commande, pas ce qu'elle fera. C'est une barrière de plus, pas une garantie. Le sandbox, lui, empêche les commandes shell d'atteindre le fichier, quel que soit leur texte.

Si ça coince

- Claude Code signale au démarrage que `settings.json` est invalide : une virgule en trop ou en moins ; faites relire le fichier par votre binôme. Dans le terminal, hors session, `claude doctor` affiche l'erreur et la clé en cause.
- Le hook ne se déclenche pas : relancez Claude Code, puis tapez `/hooks` : s'il n'y figure pas, c'est la configuration ; s'il y figure, vérifiez le matcher et le chemin du script. En dernier recours, `claude --debug=hooks`, et lisez le log dans `~/.claude/debug/`.
- Claude Code affiche une « hook error » : le script n'a pas pu se lancer ou a planté, et tant que ce message apparaît, il ne bloque rien. Relancez le test à la main (point 3 de l'étape 2) pour voir l'erreur ; sous Windows, voyez aussi le point suivant.
- Sous Windows sans Git Bash, les hooks tournent sous PowerShell : vérifiez que la commande écrit `${CLAUDE_PROJECT_DIR}` avec ses accolades. Sans accolades, PowerShell ne voit pas la variable et le chemin du script est cassé ; depuis la v2.1.299, un hook PreToolUse qui échoue ainsi peut même bloquer les actions de Claude. Si le hook échoue encore, remplacez le handler par cette forme sans shell, recommandée par la documentation. Pas de chemin relatif (`node .claude/hooks/protect-env.mjs`) : les hooks tournent dans le dossier courant de Claude, et après un `cd` le script est introuvable.

```
{
  "type": "command",
  "command": "node",
  "args": ["${CLAUDE_PROJECT_DIR}/.claude/hooks/protect-env.mjs"]
}
```

- Node.js absent : travaillez sur le poste de votre binôme.
- Le hook bloque une commande sans danger qui cite un `.env`, comme `cp .env.example .env` : lancez-la vous-même dans votre terminal.

Une fois que tout marche, remettez votre vrai `.env` en place et commitez `.claude/settings.json` et `.claude/hooks/` : toute l'équipe en profite au prochain pull.

Fichier `.claude/hooks/protect-env.mjs`

```
#!/usr/bin/env node
// protect-env.mjs : hook PreToolUse de Claude Code qui protège les fichiers
// de secrets. Node.js seul, aucune dépendance (macOS, Linux, Windows).
//
// CE QU'IL FAIT
// Avant chaque appel d'outil, Claude Code envoie sur stdin un JSON qui décrit
// l'appel (tool_name, tool_input). Le script lit les champs file_path, path,
// pattern, glob et command de tool_input. S'il y trouve un fichier de secrets,
// il écrit un message sur stderr et sort avec le code 2 : Claude Code bloque
// l'appel et transmet le message à Claude. Sinon, code 0 : l'appel continue.
//
// Bloqués  : .env et .env.<quelque chose> (.env.local, .env.production...),
//            appsettings.Development.json, appsettings.Production.json,
//            secrets.json.
// Autorisés : les fichiers d'exemple versionnés .env.example, .env.sample
//            et .env.test.
// Les chemins Windows (C:\projet\.env) sont reconnus, les majuscules aussi.
// JSON illisible sur stdin : le script laisse passer (code 0).
//
// SES LIMITES
// Le hook lit le texte de l'appel, pas ce que la commande fera vraiment.
// Une commande assez tordue passe à travers : nom de fichier fabriqué au
```

```
// moment de l'exécution, programme qui ouvre le fichier lui-même, recherche
// sur tout un dossier... C'est une barrière de plus, pas une garantie :
// le sandbox, lui, isole les commandes shell quel que soit leur texte.
// À l'inverse, il bloque des commandes sans danger qui citent un .env,
// comme « cp .env.example .env » : faites-les vous-même dans le terminal.
//
// LE TESTER À LA MAIN (le code affiché doit valoir 2)
//   bash :       echo '{"tool_input":{"command":"cat .env"}}' \
//                  | node .claude/hooks/protect-env.mjs ; echo $?
//   PowerShell : '{"tool_input":{"command":"cat .env"}}' |
//                  node .claude/hooks/protect-env.mjs ; $LASTEXITCODE
//
// L'ADAPTER : complétez les deux listes ci-dessous (noms en minuscules).

import process from "node:process";

// Fichiers d'exemple versionnés : toujours autorisés.
const EXEMPLES_AUTORISES = [".env.example", ".env.sample", ".env.test"];

// Autres fichiers de secrets (.env et .env.* sont reconnus à part).
const AUTRES_SECRETS = [
  "appsettings.development.json",
  "appsettings.production.json",
  "secrets.json",
];

// Champs inspectés : chemin d'un fichier (Read, Edit, Write), dossier et
// motifs de recherche (Grep), commande (Bash, PowerShell).
const CHAMPS = ["file_path", "path", "pattern", "glob", "command"];

// Renvoie le nom du fichier de secrets trouvé dans le texte, ou null.
function chercherSecret(texte) {
  // Découpage en noms de fichiers : lettres, chiffres, point, tiret et
  // souligné restent collés ; tout le reste sépare (espace, / et \,
  // guillemets, jokers * ?, ; | & = : ...).
  for (const nom of texte.split(/[^a-z0-9._-]+/i)) {
    const bas = nom.toLowerCase();
    if (EXEMPLES_AUTORISES.includes(bas)) continue;
    // .env seul, ou suivi d'autre chose qu'une lettre ou un chiffre :
    // .env, .env.local, .env-prod ; mais pas .environment ni .envrc.
    if (/^\.env(?![a-z0-9])/.test(bas)) return nom;
    if (AUTRES_SECRETS.includes(bas)) return nom;
  }
  return null;
}

// Lancé à la main sans rien sur stdin : on explique au lieu d'attendre.
if (process.stdin.isTTY) {
  console.error("Usage : echo '<JSON de Claude Code>' | node protect-env.mjs");
  process.exit(0);
}

let brut = "";
process.stdin.setEncoding("utf8");
for await (const morceau of process.stdin) brut += morceau;

let appel;
try {
  appel = JSON.parse(brut);
} catch {
  process.exit(0); // JSON illisible : on laisse passer.
}

const entree = appel?.tool_input ?? {};
const secret = CHAMPS.map((champ) => {
  const valeur = entree[champ];
```

```
  if (typeof valeur !== "string") return null;
  // Dans la regex d'une recherche, « \. » veut dire « un point » :
  // chercher process\.env dans le code ne lit aucun fichier .env.
  const texte = champ === "pattern" ? valeur.split("\\.").join(".") : valeur;
  return chercherSecret(texte);
}).find((nom) => nom !== null);

if (!secret) process.exit(0);

const message = [
  `Bloqué par le hook protect-env : « ${secret} » est un fichier de secrets.`,
  "Les secrets ne doivent pas entrer dans la conversation : ne cherchez pas",
  "à le lire autrement. Demandez à l'utilisateur la valeur dont vous avez",
  "besoin, ou lisez le fichier d'exemple versionné s'il existe",
  "(.env.example, appsettings.json).",
].join("\n");

// Code 2 : Claude Code bloque l'appel et transmet le message à Claude.
// On sort une fois le message écrit, pour ne pas le couper.
process.stderr.write(message + "\n", () => process.exit(2));
```

Fichier `.claude/settings.json` (`settings.exemple.json`)

```
{
  "permissions": {
    "deny": [
      "Read(**/.env)",
      "Edit(**/.env)"
    ]
  },
  "hooks": {
    "PreToolUse": [
      {
        "matcher": "Read|Edit|Write|Bash|PowerShell|Grep",
        "hooks": [
          {
            "type": "command",
            "command": "node \"${CLAUDE_PROJECT_DIR}/.claude/hooks/protect-env.mjs\""
          }
        ]
      }
    ]
  }
}
```

Bonus : un hook Stop qui garde les tests verts

Le script `tests-verts.mjs` lance vos tests quand Claude veut s'arrêter. Tests rouges : il renvoie la fin de la sortie à Claude, qui reprend pour corriger.

1. Copiez `tests-verts.mjs` dans `.claude/hooks/`.
2. Ajoutez l'événement `Stop` dans la clé `hooks` de `.claude/settings.json`, à côté de `PreToolUse`. Le fichier complet est ci-dessous ; remplacez `npm test` par votre commande de tests rapides, qui doit se terminer seule : par exemple `npm test -- --watch=false` (Angular) ou `dotnet test --filter Category=Unit` (.NET).
3. Testez le script seul : `echo '{}' | node .claude/hooks/tests-verts.mjs "npm test"`, puis `echo $?` (ou `$LASTEXITCODE` sous PowerShell). Attendu : 0 si les tests passent, 2 sinon.

Le hook ne force qu'une seule reprise : au deuxième arrêt, `stop_hook_active` est vrai et le script laisse Claude s'arrêter, même si les tests sont encore rouges. Relisez donc la fin de la session.

```
{
  "permissions": {
    "deny": [
      "Read(**/.env)",
      "Edit(**/.env)"
    ]
  },
  "hooks": {
    "PreToolUse": [
      {
        "matcher": "Read|Edit|Write|Bash|PowerShell|Grep",
        "hooks": [
          {
            "type": "command",
            "command": "node \"${CLAUDE_PROJECT_DIR}/.claude/hooks/protect-env.mjs\""
          }
        ]
      }
    ],
    "Stop": [
      {
        "hooks": [
          {
            "type": "command",
            "command": "node \"${CLAUDE_PROJECT_DIR}/.claude/hooks/tests-verts.mjs\" npm test"
          }
        ]
      }
    ]
  }
}
```

Fichier `.claude/hooks/tests-verts.mjs`

```
#!/usr/bin/env node
// tests-verts.mjs : exemple de hook Stop pour Claude Code. Node.js seul,
// aucune dépendance (macOS, Linux, Windows).
//
// CE QU'IL FAIT
// Quand Claude veut terminer son tour, Claude Code lance ce script, qui
// exécute vos tests. Tests verts : code 0, Claude s'arrête. Tests rouges :
// code 2 et la fin de la sortie sur stderr ; Claude Code empêche alors
// Claude de s'arrêter et lui transmet le message : il reprend pour corriger.
//
// La commande de tests se passe en argument ; sans argument : npm test.
//   node tests-verts.mjs                       lance npm test
//   node tests-verts.mjs npm run test:ci       lance npm run test:ci
//   node tests-verts.mjs "dotnet test --filter Category=Unit"
// Mettez la commande entre guillemets si elle contient elle-même des
// guillemets, des parenthèses ou d'autres caractères spéciaux du shell.
//
// PIÈGES
// - Tourner sans fin : si Claude continue déjà à cause d'un hook Stop
//   (stop_hook_active vrai), le script le laisse s'arrêter sans relancer
//   les tests. Conséquence : une seule relance par demande ; si les tests
//   sont encore rouges ensuite, Claude s'arrête quand même. Relisez la fin.
// - Il tourne à chaque fin de tour : réservez-le à des tests rapides.
// - La commande doit se terminer : pas de mode watch
//   (Angular : ng test --watch=false).
// - Au-delà de 5 minutes, ou si la commande ne se lance pas, il abandonne
//   sans bloquer Claude (code 1 : erreur non bloquante, affichée à l'écran).
```

```
import { spawnSync } from "node:child_process";
import process from "node:process";

const COMMANDE = process.argv.slice(2).join(" ") || "npm test";
const DELAI_MAX_MS = 5 * 60 * 1000;
const LIGNES_GARDEES = 40; // fin de la sortie renvoyée à Claude

// Lancé à la main sans rien sur stdin : on explique au lieu d'attendre.
if (process.stdin.isTTY) {
  console.error("Usage : echo '{}' | node tests-verts.mjs [commande de tests]");
  process.exit(0);
}

let brut = "";
process.stdin.setEncoding("utf8");
for await (const morceau of process.stdin) brut += morceau;

const { code, message } = verdict(brut);
if (message) process.stderr.write(message + "\n");
process.exitCode = code;

function verdict(texte) {
  let entree;
  try {
    entree = JSON.parse(texte);
  } catch {
    return { code: 0 }; // JSON illisible : on laisse Claude s'arrêter.
  }

  // Claude continue déjà à cause d'un hook Stop : on le laisse s'arrêter.
  if (entree?.stop_hook_active === true) return { code: 0 };

  const tests = spawnSync(COMMANDE, {
    shell: true,
    cwd: process.env.CLAUDE_PROJECT_DIR || process.cwd(),
    encoding: "utf8",
    timeout: DELAI_MAX_MS,
    maxBuffer: 50 * 1024 * 1024,
    windowsHide: true,
  });

  if (tests.error) {
    const raison = tests.error.code === "ETIMEDOUT"
      ? "délai de 5 minutes dépassé"
      : tests.error.message;
    return {
      code: 1,
      message: `tests-verts : « ${COMMANDE} » n'a pas abouti (${raison}).`
        + " Vérifiez la commande et qu'elle ne tourne pas en mode watch.",
    };
  }

  if (tests.status === 0) return { code: 0 };

  if (tests.status === 127) {
    return {
      code: 1,
      message: `tests-verts : commande introuvable : ${COMMANDE}`,
    };
  }

  const fin = `${tests.stdout ?? ""}\n${tests.stderr ?? ""}`
    .replace(/\x1b\[[0-9;]*m/g, "") // retire les codes couleur
    .trimEnd()
    .split(/\r?\n/)
```

```
    .slice(-LIGNES_GARDEES)
    .join("\n")
    .slice(-4000);

  const statut = tests.status ?? tests.signal;
  return {
    code: 2,
    message: [
      `Les tests ne passent pas (commande : ${COMMANDE}, code ${statut}).`,
      "Corrigez le code, pas les tests (sauf s'ils sont faux), puis",
      "relancez les tests avant de terminer. Fin de la sortie :",
      "",
      fin,
    ].join("\n"),
  };
}
```

Exercice : créer votre skill (65 min)

Phase	Durée	Ce que vous faites
0. Installer `skill-creator`	5 min	`/plugin install skill-creator@claude-plugins-official`
1. Choisir une tâche répétitive	10 min	Une procédure ou un savoir que vous recollez souvent dans le chat (idées ci-dessous).
2. Créer le skill	20 min	À la main avec le modèle ci-dessous, ou avec `skill-creator`.
3. Le déclencher sur un cas réel	15 min	Demandez la tâche sans nommer le skill. Claude l'invoque-t-il seul ? Sinon, retravaillez la description.
4. Test croisé en binôme	10 min	Votre binôme demande la tâche avec ses mots, sans nommer le skill.
5. Commit	5 min	Le dossier `.claude/skills/<nom>/` dans votre repo.

Livrable : un skill versionné dans votre repo.

Des idées par stack

Stack	Idées de skills
Angular	Composant + test ; service HTTP + mock ; revue d'accessibilité
.NET	Migration Entity Framework ; endpoint + test xUnit ; mapping DTO
Java Spring	Controller + test MockMvc
Python	Script + test pytest
Transverse	Description de PR ; changelog ; revue de code selon vos conventions
Un savoir	Le fonctionnement d'un module, sans étapes : ce qu'il faut savoir et ses pièges

Vous trouvez le code généré « peu propre » (variables inutiles, complexité excessive pour une tâche simple) ? Écrivez un skill de revue selon vos conventions, avec vos exemples de code propre.

Fiche : mon skill

Question	Votre réponse
Nom (le dossier `.claude/skills/<nom>/`)	
La tâche, en une phrase	
Quand Claude doit l'utiliser, avec les mots de vos collègues	
Le fichier modèle à suivre	
La commande qui prouve que c'est fini	
Un piège que Claude fait sur ce projet	

Mon skill se déclenche-t-il ?

- [ ] Le fichier est à `.claude/skills/<nom>/SKILL.md`, nommé exactement `SKILL.md`.
- [ ] Le frontmatter commence à la toute première ligne du fichier, entre deux lignes `---`.
- [ ] La description est entre guillemets si elle contient « : » : Claude Code le tolère, mais d'autres outils peuvent la refuser.
- [ ] Le skill apparaît quand vous tapez `/`. Sinon, et si le dossier `.claude/skills/` vient d'être créé, relancez la session.
- [ ] La description dit ce que fait le skill et quand l'utiliser, avec les mots que vos collègues emploient.
- [ ] Le skill n'a pas `disable-model-invocation: true`, sinon Claude ne peut pas le déclencher seul.
- [ ] Demandée sans nommer le skill, la tâche le déclenche : vous voyez Claude l'invoquer.
- [ ] Votre binôme l'a déclenché avec ses propres mots.
- [ ] Le `SKILL.md` reste court ; les détails sont dans des fichiers annexes.
- [ ] Le skill est commité.

Le skill n'apparaît pas : vérifiez le chemin et le nom du fichier ; si le dossier `.claude/skills/` a été créé après le lancement de la session, `/reload-skills` ou une nouvelle session. Le skill apparaît mais ne se déclenche jamais : `claude plugin validate .claude/skills` signale un frontmatter invalide (le skill se charge alors sans sa description), comme le journal de `claude --debug`. (`/skill-doctor` sert à autre chose : le coût et l'usage des skills déjà chargés.)

Modèle : `SKILL.modele.md`

```
---
# Modèle de SKILL.md. Rangez-le dans .claude/skills/<nom>/SKILL.md et
# supprimez ces lignes de commentaire une fois le modèle rempli.
#
# name : nom du skill, celui qu'on tape après « / ». Minuscules et tirets.
# Par défaut, Claude Code prend le nom du dossier : gardez les deux identiques.
name: nom-du-skill
#
# description : ce que fait le skill, puis quand l'utiliser. C'est elle qui
# déclenche le skill : écrivez-la avec les mots de vos collègues. Gardez-la
# entre guillemets : Claude Code tolère un « : » dans la description, mais
# d'autres outils peuvent la refuser, et un YAML invalide charge le skill
# sans aucun de ses champs.
description: "Fait <la tâche> selon nos conventions. À utiliser quand <situation>."
#
# when_to_use : facultatif. Phrases de déclenchement, exemples de demandes.
# Ajouté à la description dans la liste des skills (1 536 caractères au
# total pour les deux).
when_to_use: "Exemples : « <demande typique 1> », « <demande typique 2> »."
---

# <Titre du skill>

## Objectif

<!-- Une ou deux phrases : le résultat attendu, et pour qui. -->

## Étapes

<!-- Numérotées, une action par ligne, avec les commandes exactes du projet
     et le fichier modèle à suivre. -->

1. Lire `<fichier modèle à suivre>`.
2. Lancer `<commande exacte>`.
3. Écrire `<fichier>` en suivant le modèle.
4. Lancer `<commande de test>` et corriger jusqu'à ce qu'elle passe.

## Pièges

<!-- Ce que Claude fait de travers sur CE projet. Une ligne par piège.
     Ajoutez-en un chaque fois que le skill se trompe. -->

- Ne pas <erreur fréquente> : <ce qu'il faut faire à la place>.

## Vérification

<!-- Comment Claude prouve que c'est fini : commande, sortie attendue. -->

- `<commande de test>` passe.
- Le diff ne touche que <périmètre attendu>.
```

Le ticket fil rouge

Ce soir, choisissez dans votre backlog le ticket qui servira demain : à l'Atelier 2, vous en écrirez la spec et le plan d'exécution ; à l'Atelier 3, vous l'exécuterez en parallèle. Demain matin, vous le dites en une phrase, 15 secondes chacun.

Pas de projet accessible depuis votre poste de formation ? Demain, la pratique se fait aussi sur Tailspin Toys, un projet d'entraînement commun : vous y choisirez votre ticket sur place, dans son backlog ou parmi vos idées. Rien à préparer ce soir.

Critère	La question à se poser	Mon ticket
Taille	Une demi-journée à deux jours de travail pour un humain ?	oui · non
Découpable	Peut-il se couper en 2 ou 3 morceaux indépendants ?	oui · non
Vérifiable	Des tests diront-ils que c'est fini ? Sinon, l'agent écrira les tests d'abord.	oui · non
Montrable	Pas confidentiel au point de ne pas pouvoir en parler devant le groupe ?	oui · non

Gardez le deuxième ticket candidat en réserve. Si aucun ne convient, prenez demain un ticket de Tailspin Toys.

Fiche : mon ticket fil rouge (à remplir ce soir)

Question	Votre réponse
Projet	
Ticket : titre et lien	
Ce qui change pour l'utilisateur, en une phrase	
« Fini » veut dire… (un critère vérifiable)	
La commande qui le vérifie	
Morceau 1	
Morceau 2	
Morceau 3 (facultatif)	
Ce que les morceaux partagent (fichiers, types, données)	
Les fichiers ou les zones touchés	
Des tests existent sur cette zone ?	oui · non · en partie
Ce qui m'inquiète, les inconnues	
Mon ticket de réserve	

J2 matin — Cahier

Le fil rouge du J2 : un seul ticket, du matin au soir.

Quand	Étape	Ce que vous produisez
En arrivant	Votre projet, ou Tailspin Toys	Un projet qui tourne, la chaîne de skills installée
Début de matinée	Le projet présenté par Claude, puis votre ticket du jour	Votre ticket, en une phrase
Milieu de matinée	Atelier 2, étape par étape	La spec, les tickets, le plan de composition
Fin de matinée	« `/goal` à blanc », puis la condition d'arrêt	Une condition `/goal`, écrite puis lancée : la loop tourne pendant le déjeuner
Début d'après-midi	La préparation des worktrees	Votre branche, prête pour l'Atelier 3
Après-midi	Atelier 3, puis l'exercice MCP	Les tickets en parallèle, un serveur MCP et son sous-agent

Votre projet, ou Tailspin Toys

Un projet sur lequel avancer aujourd'hui, perso ou client, accessible depuis ce poste ? Restez dessus. Sinon, installez Tailspin Toys, le projet d'entraînement de la journée :

```
git clone https://github.com/denneulin/tailspin-toys.git
cd tailspin-toys
npm ci
npm run dev        # http://localhost:4321
```

Git, Node 24, un poste sans droits d'administrateur, le partage d'écran : tout est sur la page « Démarrer le J2 », à côté du cahier sur la page du livret.

Installer la chaîne

Puis, pour tous, dans Claude Code lancé dans votre projet : la marketplace de Matt Pocock, et elle seule aujourd'hui, pour que tout le monde ait la même version (affichée 1.3.1).

```
/plugin marketplace add mattpocock/skills
/plugin install mattpocock-skills@mattpocock
```

Sur Tailspin, le setup est déjà fait. Sur votre projet, lancez `/setup-matt-pocock-skills` sur une branche, et choisissez les fichiers Markdown locaux : il écrit dans `docs/agents/` et ajoute un bloc « ## Agent skills » à votre CLAUDE.md.

Exercice : faites-vous présenter le projet (7 min)

1. **À votre façon (4 min)**. Lancez Claude Code dans le projet, et demandez-lui de vous le présenter : ce que vous demanderiez à un collègue, le premier jour. **Ne modifiez rien.**
2. **Puis ce prompt (3 min)**, dans la même session. Sur votre projet, remplacez `backlog/` par votre outil de tickets.

```
Ne modifie rien. Présente-moi ce projet en cinq lignes : ce qu'il fait,
sa stack, comment on lance les tests. Puis résume chaque ticket de
backlog/ en une ligne : ce qui change pour l'utilisateur.
```

Question	Votre réponse
Ce que le prompt a changé, par rapport à votre façon	

Gardez la réponse ouverte : vous choisissez votre ticket juste après.

Votre ticket du jour (5 min)

Votre ticket	Où le trouver
Un ticket de votre projet, perso ou client	votre backlog, avec les quatre critères de la fiche d'hier soir
La 07, proposée à tous sur Tailspin : filtrer les jeux par catégorie et par éditeur	`backlog/07-filter-games.md`
Un autre ticket du backlog de Tailspin, de 01 à 06	`backlog/` et le README
Votre idée sur Tailspin : une liste de souhaits, des avis…	une phrase à vous

Mon ticket, en une phrase : ce qui change pour l'utilisateur, pas comment le coder

Puis tour rapide : 15 secondes chacun, votre phrase.

Si ça coince : la 08 (des standards de code, rien de visible à l'écran) et la 09 (un assistant IA, il faut une clé d'API) sont déconseillées. Une idée trop petite (un libellé, une couleur) : il n'y aurait rien à découper ni à paralléliser ; élargissez-la, ou prenez la 07. Une idée qui enregistre quelque chose (une liste de souhaits, des avis) passe par une route servie à la demande : le README donne le modèle, `src/pages/api/games/[id].ts`. Un ticket de votre projet trop gros : n'en gardez qu'une tranche, qui se livre et se vérifie seule. Un ticket confidentiel : décrivez-le en termes génériques.

Atelier 2 : de votre ticket à une spec et des tickets (58 min, seul)

Chacun sur son ticket, seul. On avance ensemble, étape par étape : chaque étape s'affiche quand on y arrive, et on passe à la suivante au signal. Fini en avance ? Chaque étape dit quoi approfondir.

Livrable : la spec de votre ticket, vos tickets avec leurs dépendances, et votre plan de composition. C'est le livrable 3 du programme (un plan de composition de skills sur une tâche réelle), et la carte de l'Atelier 3.

Étape 1 : se faire interviewer (23 min)

```
git switch -c <votre-ticket>     # 07-filtres pour la 07
/model opus
/grill-with-docs <votre ticket : le chemin du fichier, ou votre phrase>
```

- **La branche** porte le nom de votre ticket : dès l'interview, Claude écrit dans le repo, et tout ce que vous produisez aujourd'hui va sur cette branche, votre branche de travail. Sur votre projet, si vous avez déjà créé une branche pour le setup, restez dessus.
- `/model opus` : c'est déjà le modèle par défaut, on le vérifie, parce que la conception, c'est du jugement.
- **Tranchez les termes** qu'il vous propose : ils partent dans `GLOSSARY.md`, à la racine du repo.
- **Répondez vraiment** quand une recommandation ne vous va pas : le product owner, c'est vous. Celles qui vous vont, acceptez-les en le disant en toutes lettres : « j'accepte tes recommandations pour Q1 et Q2 ».
- Quand il résume et vous demande s'il peut y aller : **ne dites pas oui**. On passe à l'étape suivante ensemble.

Fini en avance ? Demandez-lui : « Quelles décisions as-tu prises à ma place ? », puis relisez `GLOSSARY.md` : chaque terme a-t-il un seul sens ?

Étape 2 : la spec (9 min)

Dans la même session, sans `/clear` :

```
/to-spec Dis-moi d'abord où tu comptes tester, et attends mon accord avant d'écrire.
```

- Il **n'interviewe plus** : il synthétise votre conversation, explore le code et vous propose ses **points de test** (*seams*), là où ses tests attaqueront le code : une fonction, une route, une page. Validez-les ou corrigez-les.
- Puis il écrit `spec.md` dans `.scratch/<fonctionnalité>/`. Lisez surtout **Implementation Decisions** et **Out of Scope**.

Fini en avance ? Relisez Out of Scope : ce qui y manque, faites-le ajouter (« Ajoute à Out of Scope : … »).

Étape 3 : les tickets (11 min)

Toujours dans la même session :

```
/to-tickets
```

- Demandez **un premier ticket qui en débloque deux** : une loop fera le premier avant le déjeuner, les deux suivants partiront en parallèle l'après-midi. Sur la 07, le ticket le prévoit déjà : vérifiez-le.
- **Refusez un découpage par couche** : une tranche verticale par ticket, vérifiable seule.
- Les tickets arrivent dans `.scratch/<fonctionnalité>/issues/`, chacun avec sa ligne « Blocked by ».

Relisez vos tickets avant l'étape 4 : personne ne le fera à votre place.

- [ ] Chaque ticket est une tranche verticale : il traverse toutes les couches.
- [ ] Chacun se vérifie seul.
- [ ] Les « Blocked by » sont justes.

Étape 4 : le plan de composition (9 min)

Une ligne par ticket. Recopiez « Bloqué par » depuis vos fichiers de tickets. Notez les skills qui existent déjà et serviront (ceux de la chaîne comme `/tdd` et `/implement`, celui que vous avez écrit au J1 s'il s'applique, ceux de votre projet), puis ceux qui manquent, et la commande qui prouve que le ticket est fini.

Ticket	Bloqué par	Skills qui existent	Skills à créer	Vérification
Exemple : 01 Filtrer par une catégorie, de bout en bout	—	`/tdd`, `/implement`	*un skill « filtre de liste », si ça revient*	`npm run test:unit`

- **Bloqué par** : le ticket qui n'est bloqué par rien part dans une loop avant le déjeuner ; ceux qu'il bloquait partent en parallèle l'après-midi, un worktree chacun.
- **Vérification** : le point de départ de votre condition d'arrêt. Dans Tailspin, le CLAUDE.md la donne : les tests, le lint et les types (`npm run test:unit`, `npm run lint`, `npm run typecheck`). Sur votre projet, la commande de test en mode CI de l'équipe. Pas de tests sur la zone : écrivez « d'abord : un test qui reproduit le comportement attendu ». C'est une vraie ligne de plan, et souvent un premier ticket.
- **Skills à créer** : un skill existant suffit-il ? Une étape qui se répète assez pour mériter un skill (générer des données de test, ajouter un filtre ou un tri à une liste, créer un composant au gabarit de l'équipe, écrire une route) est un bon candidat pour votre plan d'adoption à 30 jours.

Restitution express (5 min)

Trois volontaires, 90 secondes chacun : votre ticket en une phrase ; combien de tickets, et lesquels peuvent partir en parallèle ; une question de l'interview qui vous a surpris.

Si ça coince

- L'installation coince encore : demandez de l'aide d'abord ; l'interview n'a besoin que du projet et des skills.
- `/model opus` refusé, ou un autre modèle par défaut : restez sur le modèle proposé, ce n'est pas bloquant.

- Pas de `GLOSSARY.md` : aucun terme n'a encore été tranché ; il le crée au premier.
- Une ADR proposée dans `docs/adr/` : une décision lui semble difficile à défaire. Sur la 07, la plus probable : filtrer dans le navigateur, sur la liste déjà affichée, ou servir la liste à la demande. Les deux se défendent : c'est à vous de trancher.
- L'interview n'en finit pas : c'est normal, le skill pose des dizaines de questions. Au signal, on passe à l'étape 2 quoi qu'il reste : la synthèse se contente de ce qui a été dit, et signale ce qui n'a pas reçu de réponse.
- Claude se met à coder à la fin de l'interview : Esc, puis `/rewind` pour revenir au point d'avant (il ne suit pas les modifications faites par des commandes Bash), ou « Annule ces modifications ». Vérifiez avec `git status`.
- `/to-spec` demande de lancer le setup : sur Tailspin, vérifiez que la session tourne dans le dossier `tailspin-toys` ; sur votre projet, lancez `/setup-matt-pocock-skills`, choisissez les fichiers locaux, puis relancez `/to-spec`.
- `/to-spec` dit avoir suivi ses recommandations pour les questions restées sans réponse : il ne cache pas les trous ; relisez ce passage.
- Spec très longue : c'est le gabarit. Lisez Implementation Decisions et Out of Scope.
- Trop de tickets (six ou sept pour la 07) : « Fusionne en quatre tickets au plus. »
- Tickets par couche (« 01 la fonction de filtrage, 02 l'interface ») : refusez. « Une tranche verticale par ticket, testable seule. »
- Sur la 07, le ticket a une section « Delivery » : une catégorie de bout en bout, puis l'éditeur et plusieurs catégories en parallèle, chacun dans ses fichiers, puis la combinaison, en tranche à part. Votre graphe doit donc être 01, puis 02 et 03 en parallèle, puis 04.
- Sur la 07, pas deux tickets à lancer en parallèle après le premier, ou pas de 04 : « Relis la section Delivery du ticket 07, et redécoupe en la suivant. » Si ça résiste, le découpage de secours a la bonne forme : `git fetch origin`, puis `git switch secours/07-filtres`.
- Graphe linéaire alors que le ticket semblait découpable : demandez à `/to-tickets` pourquoi. Parfois c'est justifié (un refactoring large) ; parfois, un ticket préalable qui rend le changement facile débloque tout le reste.
- Le setup a modifié votre CLAUDE.md : c'est normal, et c'est pour ça que vous êtes sur une branche.
- Java, Python ou .NET : la chaîne ne dépend pas du langage.

Exercice : « `/goal` à blanc » (4 min)

Sur votre branche, d'abord `/clear` : sans lui, Claude a vos tickets en tête et pourrait se mettre à coder. Puis :

```
/clear
/goal npm run test:unit passe ; sinon arrête-toi après 2 tours
```

Sur votre projet : votre commande de tests, à la place de `npm run test:unit`. Regardez combien de tours il lui faut, et ce qu'il a écrit. Puis Ctrl+O : la raison du verdict.

Question	Votre réponse
Combien de tours, et ce qu'il a écrit	
Qu'est-ce qui cloche ?	

Si ça coince : Claude se met à coder un ticket (le `/clear` oublié) : Esc, `/goal clear`, `git status`, puis recommencez avec `/clear`. Sur votre projet, sans tests ou avec une commande qui échoue : l'évaluateur rend « pas encore », et le plafond arrête Claude ; c'est l'autre leçon.

Exercice : la condition d'arrêt de votre ticket (11 min)

1. Prenez le **premier ticket** de votre graphe : celui qui n'est bloqué par rien.
2. Écrivez sa **condition** `/goal` ci-dessous : un état final mesurable, une vérification énoncée, les contraintes, **et un plafond**. Partez de la colonne « Vérification » du plan de composition et des critères d'acceptation du ticket : chaque case à cocher vérifiable peut devenir une clause. Ce qui manque au harness pour lâcher ce ticket, mettez-le dans la condition.
3. Relisez-la avec la question et la liste ci-dessous. **Ne la lancez pas encore** : on la lance ensemble juste après. Fini en avance ? Écrivez celle du ticket suivant de votre graphe, dans la dernière case : elle servira cet après-midi.
4. **Au signal, lancez-la** : sur votre branche, vérifiez le mode Auto (▸▸ auto mode on dans la barre d'état ; sinon, Shift+Tab), puis collez votre condition, `/goal …`. `/goal` seul affiche le statut ; Ctrl+O, la raison du verdict. Elle tourne pendant le déjeuner : session ouverte, ordinateur branché, sans mise en veille, et pas de `/clear`. Le verdict, on le regarde à la reprise.

```
/goal <le nouveau test du ticket, nommé> passe,
  et npm run test:unit, npm run lint et npm run typecheck passent ;
  aucun autre fichier de test n'est modifié, ce que montre git status ;
  sinon arrête-toi après <n> tours.
```

Sur votre projet : votre commande de tests, de lint ou de build, à la place des trois commandes de Tailspin.

	Votre réponse
Le ticket (le premier de votre graphe)	

Ma condition `/goal`

La question de la relecture	Ma réponse
« L'évaluateur pourrait-il trancher en lisant seulement la sortie de Claude ? »	oui · non
Ce que je change	

À la répétition, l'évaluateur a validé une condition en notant « …but the condition does not require that rendering check ». Il juge ce que vous avez écrit, pas ce que vous vouliez dire.

Relecture : ma condition a-t-elle tout ?

- [ ] Un état final mesurable : un résultat de tests, un code de sortie, un nombre de fichiers.

- [ ] Une vérification énoncée : la commande dont la sortie montre cet état.
- [ ] Elle est fausse avant le travail : elle nomme le test ou le critère du ticket, pas seulement une suite qui passe déjà (c'est le piège de « `/goal` à blanc »).
- [ ] Les contraintes qui comptent, dont : aucun autre fichier de test n'est modifié, ce que montre `git status`.
- [ ] Un plafond : « sinon arrête-toi après N tours ».
- [ ] Tout est visible dans la sortie de Claude : une contrainte comme « ne touche pas à la page d'accueil » se démontre, par exemple avec un `git status` demandé dans la condition.

Fini en avance : la condition du ticket suivant

Les pièges fréquents

- « Le code est propre », « le filtre fonctionne » : invérifiables. Remplacez-les par une commande de lint qui sort en 0, ou par un test qui prouve le filtre et la commande qui le lance. L'écran, c'est vous qui le regardez : l'évaluateur ne lit que la sortie de Claude.
- Condition déjà vraie avant le travail : la suite est verte et la condition ne nomme pas le test du ticket. L'évaluateur dira « atteint » au premier tour, sans une ligne écrite : nommez le test ou le critère du ticket.
- Pas de test sur la zone : la condition commence par « un test qui reproduit le comportement attendu est écrit et échoue, puis passe ». Ticket « expand », qui ajoute sans rien changer à l'écran : elle porte sur les nouveaux tests, et sur les anciens qui restent verts.
- Suite de tests trop longue (plusieurs minutes) : ciblez les tests du module, puisque chaque tour relance la commande.
- La loop lancée en mode Manual : elle attendrait une réponse à la première demande de permission, pendant tout le déjeuner. Esc, `/goal clear`, passez en Auto, et relancez-la.

J2 après-midi — Cahier

Exercice : préparez vos worktrees (5 min)

Un worktree ne contient que ce qui est committé : sans ce commit, il ne voit ni vos tickets ni le code de votre loop.

1. **Committez** sur votre branche de travail ce que le matin a produit : le `.scratch/` de votre ticket, `GLOSSARY.md`, `docs/adr/` s'il existe, et le code de votre loop.
2. **Vérifiez** : `git status` est propre, et `git log -1` montre ce commit.
3. **Créez votre premier worktree** : `claude -w <ticket> --model sonnet`, vérifiez qu'il voit vos tickets (`!ls .scratch`), puis `/exit` : l'étape 1 de l'atelier le recrée sur le dernier commit. Ne lancez pas encore d'agent.

Dans Tailspin, vos worktrees partent de votre commit, et rien n'est à installer dedans. La loop de midi a échoué sur la 07 : `git fetch origin`, puis `git switch secours/07-filtres-01-fait`.

Sur votre projet, trois gestes de plus :

- [ ] `.claude/worktrees/` dans le `.gitignore`.
- [ ] `{ "worktree": { "baseRef": "head" } }` dans `.claude/settings.local.json` : sans lui, le worktree part de la branche par défaut du remote, sans vos tickets ni le setup. Si le réglage résiste : `git worktree add -b <branche> <dossier>`, puis `claude` dans ce dossier.
- [ ] Les dépendances : `npm ci` côté Angular, `dotnet restore` côté .NET, ou demandez à Claude de les installer ; le `.env`, copié à la main pour l'atelier (`.worktreeinclude` pour la suite).

Atelier 3 : vos tickets, en parallèle (84 min, pause comprise, seul)

Chacun sur ses tickets, seul. Comme à l'Atelier 2, on avance ensemble, étape par étape, et on change d'étape au signal. La pause de 15 minutes est fixe : la loop du ticket suivant tourne pendant ce temps.

Livrable : deux tickets faits en même temps, puis fusionnés ; une analyse en parallèle ; vos tentations, classées. C'est le livrable 4 du programme : un workflow multi-agent testé avec worktrees et sous-agents.

La règle de tout l'atelier : tant que le harness ne remonte rien, vous n'intervenez pas. Chaque tentation de reprendre la main, notez-la, avec sa raison, dans la grille des tentations, juste après l'atelier : on les classe à l'étape 5.

Ce qui compte comme « le harness remonte quelque chose » :

- Une demande de permission : vous y répondez, c'est autorisé.
- Un hook qui bloque.
- Un verdict « impossible » de `/goal`.
- Le plafond de tours atteint.
- Une session qui rend la main avec un « reste à faire ».

Tout le reste, y compris une session qui semble partir dans le décor : notez la tentation, n'intervenez pas.

	Votre réponse
Les deux tickets de ma frontière : ceux qui ne sont plus bloqués par rien	
Le ticket qu'ils débloquent	

Étape 1 : deux tickets, deux worktrees, en même temps (11 min)

Deux terminaux, à la racine de votre projet, un par ticket de votre frontière.

```
# Terminal 1 : le skill de Pocock
claude -w <ticket-1> --model sonnet
/implement <chemin du ticket 1>

# Terminal 2 : votre pipeline
claude -w <ticket-2> --model sonnet
<le prompt du pipeline, avec le chemin du ticket 2>
```

- ▸▸ **auto mode on** dans chaque session (sinon, Shift+Tab), et **lancez les deux** avant d'en regarder un : c'est ça, le parallèle.
- À regarder : le test d'abord, les sous-agents de revue (`/tasks`), le commit de `/implement` ; le « fait / reste à faire » du pipeline, qui ne committe pas.
- L'implémentation tourne sur Sonnet ; dans le pipeline, seul le relecteur passe sur Opus.

Un seul ticket sur la frontière, parce que le graphe est linéaire ? Le pipeline seul, dans un worktree : c'est déjà un workflow multi-agent. Fini en avance ? Votre condition `/goal`, adaptée au ticket, dans chaque session, quand Claude a rendu la main : elle doit nommer le nouveau test, puisque les tests existants passent déjà.

	Votre réponse
Le nom de mes deux worktrees	

Le prompt du pipeline. Remplacez le chemin par celui de votre ticket 2. Il est expliqué sur la slide « Un pipeline, en un prompt » ; le détail est dans le récap.

```
Prends le ticket .scratch/<fonctionnalité>/issues/01-….md.
Attends le rapport de chaque sous-agent avant l'étape suivante.
1. Implémentation : un sous-agent Sonnet nommé impl. Il ne lance
   pas d'agent. Il lance les tests ciblés avant son rapport ;
   un critère d'acceptation = une preuve.
2. Revue : un sous-agent Opus, en lecture seule, relit le diff
   contre le ticket, sans le rapport d'impl. Il ne corrige rien
   et ne lance pas d'agent. Il ne signale que ce qui touche la
   correction ou le ticket : bloquant / important / mineur,
   avec fichier:ligne.
3. Correction : renvoie les constats à impl avec SendMessage,
   un tour au plus.
4. Ne committe pas. Termine par : fait / reste à faire.
```

Étape 2 : relire, contre le ticket (12 min)

Dans chaque session (le point d'exclamation lance la commande vous-même ; `git add -N` fait entrer dans le diff les fichiers créés par le pipeline, sans les indexer) :

```
!git show --stat            # /implement : il a committé
!git add -N . && git diff   # le pipeline : rien n'est committé
```

- **Le ticket ouvert à côté**, critère par critère : qu'est-ce qui le prouve ? Le diff et le test, pas le rapport de l'agent.
- **Le site** : « Lance le serveur de dev et donne-moi l'URL », dans chaque session ; un port chacun (4321, 4322…). Faites dans le navigateur ce qu'aucun test ne prouve : sur la 07, la vérification au clavier que demandent les tickets.

- **Le travail du pipeline** que vous gardez : « Committe ce ticket sur cette branche. » Sans commit, l'étape 3 ne fusionne rien.
- Un défaut trouvé : notez-le dans la grille, puis renvoyez-le à la session (« Le critère 3 n'est pas prouvé : … ») ; ne le corrigez pas vous-même dans l'éditeur.

Fini en avance ? Vérifiez dans le code un constat du relecteur Opus. Et lequel a trouvé le plus : la revue de `/implement`, ou le pipeline ?

Question	Votre réponse
Un critère qu'aucun test ne prouvait, et ce que j'ai vu dans le navigateur	
La revue qui a trouvé le plus : `/implement` ou le pipeline	

Étape 3 : fusionner, la frontière avance (13 min)

Un troisième terminal, à la racine, sur votre branche de travail :

```
git merge --no-edit worktree-<ticket-1>
git merge --no-edit worktree-<ticket-2>
npm run test:unit
```

- **Un conflit ?** Dans ce terminal, `claude`, puis « Résous le conflit, et lance les tests ». Vous relisez. Notez-le dans la grille : le découpage n'était pas si indépendant.
- **Le ticket suivant est débloqué** : écrivez sa condition `/goal` ci-dessous, sur le modèle de ce matin, puis, dans un quatrième terminal, à la racine, `claude -w <ticket-3> --model sonnet`, le mode Auto vérifié, et lancez-la. Elle tourne pendant la pause : session ouverte, ordinateur branché, pas de mise en veille.
- Le piège, quand les deux fusions suffisent à faire marcher le ticket suivant (sur la 07, la combinaison) : « les tests passent » est vrai avant le premier tour, c'est le « `/goal` à blanc » de ce matin. La condition nomme le nouveau test.

Rien de débloqué ? Une condition `/goal` sur ce qui reste de votre graphe.

La condition `/goal` du ticket débloqué

Pause (15 min)

Votre loop tourne. Sur votre projet, et vous voulez faire l'exercice MCP sur Tailspin ? Installez-le maintenant (page « Démarrer le J2 »).

Étape 4 : analyser en parallèle (17 min)

1. **Le verdict de votre loop**, dans le terminal 4 : `/goal`, puis Ctrl+O. Atteinte : relisez son diff (`!git add -N . && git diff`), puis « Committe ce ticket sur cette branche ». Pas atteinte : la raison de l'évaluateur va dans la grille ; c'est souvent la condition, pas l'agent, et on ne fusionne pas.

2. **Terminal 3**, à la racine : si la loop est atteinte, `git merge --no-edit worktree-<ticket-3>` puis `npm run test:unit` ; l'analyse portera ainsi sur le code complet. Puis `claude --model sonnet` (les sous-agents Explore suivent le modèle de la session), et le prompt ci-dessous, rempli. Sur Tailspin : le module `src`, les axes tests, accessibilité, sécurité, le nom `src`. Regardez `/tasks` : trois sous-agents, en même temps, en lecture seule.
3. **Vérifiez vous-même** une affirmation de `roadmap.md` que l'orchestrateur n'a pas vérifiée, en ouvrant le fichier cité.

`<module>` est le dossier à auditer, `<axe 1>` à `<axe 3>` vos axes, `<nom>` un nom court pour le dossier des rapports. Avec quatre axes, écrivez « quatre » partout où le prompt dit « trois ». Restez en mode Auto, ou passez en Accept edits : l'orchestrateur écrit quatre fichiers.

```
Audite le module <module> sur trois axes, en parallèle : <axe 1>, <axe 2>, <axe 3>. Lance trois sous-agents Explore (lecture seule), un par axe, en même temps.

Brief de chaque sous-agent :
- Objectif : les 3 problèmes les plus importants sur ton axe, classés par impact.
- Sortie : pour chaque problème, le constat, l'impact, fichier:ligne et la correction proposée.
- Limites : reste dans <module> et sur ton axe ; ne modifie rien ; ne lance pas d'agent.
- Cite un chemin pour chaque affirmation ; marque NON VÉRIFIÉ ce que tu n'as pas vérifié.

Attends les trois rapports. Écris-les dans .scratch/audit-<nom>/<axe>.md, puis fais la synthèse dans .scratch/audit-<nom>/roadmap.md : doublons fusionnés, les trois premières actions. Vérifie toi-même une affirmation de chaque rapport en ouvrant le fichier cité, et dis laquelle.
```

Sur votre projet : votre module, vos axes ; la sécurité, l'observabilité ou la performance sont des axes possibles. Fini en avance ? Vérifiez une deuxième affirmation, ou ajoutez un quatrième axe : performance.

	Votre réponse
Le verdict de ma loop, et sa raison	
Le module et les axes que j'audite	
L'affirmation de la roadmap que j'ai vérifiée : vraie ou fausse ?	

Étape 5 : classer vos tentations (15 min)

La grille des tentations, juste après l'atelier : une ligne par tentation. Entourez la correction la plus utile : elle ira dans la ligne 4 de votre plan d'adoption. Les cinq dernières minutes, on range : `/exit` dans chaque session (« Keep » si Claude Code demande s'il faut garder un worktree dont la branche n'est pas fusionnée), `npx astro dev stop` là où tourne un serveur de dev, et `git worktree list` pour voir ce qui reste. Le terminal 3 reste ouvert, à la racine : l'exercice MCP s'y fait.

Le repli : l'analyse parallèle, dès l'étape 1

Sur votre projet, un ticket qui ne se découpe pas, ou pas de tests sur la zone : faites l'analyse parallèle de l'étape 4 dès l'étape 1, sur votre repo, en changeant le module et les axes. À l'étape 4, ajoutez un quatrième axe, ou vérifiez deux affirmations. Sur Tailspin, le repli, c'est le découpage de secours de la 07 : 01 une catégorie, de bout en bout ; 02 l'éditeur et 03 plusieurs catégories, en parallèle ; 04 la combinaison (branches `secours/*` du fork).

Si ça coince

- `claude -w` refuse : pas de dépôt git, ou dossier jamais approuvé. Lancez `claude` une fois à la racine du repo et acceptez le dialogue de confiance (son curseur est sur « No, exit »).
- Le worktree ne voit pas vos tickets : ils n'étaient pas committés ; ou, sur votre projet, `baseRef` manque. Reprenez la préparation.
- Dépendances : dans Tailspin, rien à installer dans un worktree de `claude -w`, il trouve le `node_modules` du checkout principal ; `npm ci` seulement si le ticket ajoute une dépendance.
- Une demande de permission pour `npm run build` : il n'est pas dans les commandes autorisées de Tailspin. En Auto, il passe ; ailleurs, acceptez.
- Le pipeline rend la main avec deux options à trancher : c'est le harness qui remonte quelque chose. Tranchez, puis relancez.
- Une session veut lancer Playwright : le CLAUDE.md de Tailspin l'écarte, faute de navigateur de test. Une tentation à noter.
- `/implement` annonce une revue qui ne trouve rien, ou relit un ticket précédent : son `/code-review` ne voit que ce qui est committé. Faites committer, puis relancez `/mattpocock-skills:code-review`. Tentation à classer : harness.
- `/goal` répond « atteint » dès le premier tour alors que le ticket n'est pas fini : la condition était vraie sans le travail du ticket. C'est un trou de harness, pas une erreur de l'agent : la condition doit nommer le nouveau test ou le critère du ticket.
- `git merge` ouvre un éditeur (Vim, souvent) : `:wq`, puis Entrée. C'est ce que `--no-edit` évite.
- Deux sessions veulent le même port : Astro prend le port libre suivant ; `npx astro dev stop` arrête un serveur. Sur un projet .NET avec des tests d'intégration sur une base locale : en parallèle, seulement les tests unitaires.
- Limites d'usage atteintes : chaque session et chaque sous-agent dépensent leurs propres tokens. Deux sessions au plus, pas quatre : le pipeline lance déjà deux sous-agents par ticket, dont un relecteur sur Opus.
- La liste de `/tasks` s'allonge, des sous-agents en lancent d'autres (souvent pendant le `/code-review` de `/implement`) : c'est la cascade. Intervenez : `Ctrl+X Ctrl+K`, deux fois en trois secondes, arrête les sous-agents en arrière-plan (Échap seul ne les arrête pas), puis notez la tentation : un trou de harness, le brief n'interdisait pas de lancer des agents.
- Le pipeline part en agent team au lieu de sous-agents : `CLAUDE_CODE_EXPERIMENTAL_AGENT_TEAMS` est à 1 ; remettez-la à 0 et relancez la session.
- Vous avez tapé `/model sonnet` au lieu de l'option `--model` : ça marche, mais c'est devenu le défaut de vos sessions suivantes. Repassez sur Opus avant de concevoir votre prochain ticket.
- Vous préférez `/implement` pour le ticket débloqué : marquez d'abord ses bloqueurs comme fermés (leur ligne `Status:` dans `.scratch/`), committés sur la branche de travail, puis `claude -w`.

La grille des tentations

Pour chaque tentation d'intervenir : la tentation, ce qui a cloché (entourez la classe), ce que vous corrigez. Pour chaque ligne, posez-vous la question : « Qu'est-ce qui aurait empêché l'erreur ? » S'il y a une réponse, ce n'est pas une erreur d'agent.

Ma tentation	Ce qui a cloché	Ce que je corrige
Exemple : « Le relancer : il a modifié un test »	*harness*	*« aucun test n'est modifié » dans la condition, ou un hook*
Exemple : « Lui expliquer le format de date attendu »	*spec*	*une ligne dans la spec, puis relancer*
	harness · spec · code · agent	
	harness · spec · code · agent	
	harness · spec · code · agent	
	harness · spec · code · agent	
	harness · spec · code · agent	
	harness · spec · code · agent	

Les quatre classes, celles de « Quand vous rentrez dans la boucle » : **harness** (une vérification manquait) · **spec** (elle avait un trou) · **code** (il résiste : module trop couplé, pas de point d'entrée testable) · **agent** (une vraie erreur de l'agent : relancer, découper plus fin). On se classe trop vite en « erreur d'agent », parce que c'est la seule classe qui ne demande rien de nous.

On corrige la cause, pas la sortie. Sur une tâche unique, corriger la sortie est légitime ; corriger la cause ne paie que sur ce qui se répète.

La correction la plus utile, pour la ligne 4 du plan d'adoption

Exercice : un serveur MCP, et le sous-agent qui s'en sert (45 min)

Quatre étapes, au même rythme que l'atelier, dans le troisième terminal, à la racine du projet, pas dans un worktree. Sur votre projet : un serveur en lecture seule sur une donnée simple (un fichier de données, une table de dev en lecture, une API locale), en adaptant le prompt ; le SDK officiel existe aussi en C# et en Java. Rien de simple à brancher : l'exercice sur Tailspin.

Dans l'app desktop (onglet Code)

- **La session** : *Local*, sur le dossier de Tailspin, **sans** l'option worktree ; Sonnet dans le menu du modèle, à côté du bouton d'envoi. La branche : `git switch -c mcp-catalogue` dans le terminal intégré (`Ctrl+`).
- **Avant l'étape 1**, demandez à Claude : « Lance npx tsx --version. » S'il ne trouve pas `npx` : ajoutez le dossier de Node au `Path` de votre compte (menu Démarrer, « Modifier les variables d'environnement pour votre compte »), puis quittez et relancez l'app.
- **Étape 2** : la commande `claude` n'existe pas hors de l'app. Demandez à la place : « Enregistre ce serveur pour le projet dans .mcp.json, à la racine : nom tailspin, commande npx, arguments tsx et mcp/catalog-server.ts. » Puis une nouvelle session (Ctrl+N), sur le même dossier, *Local*, sans worktree. Si un dialogue demande d'approuver le serveur, choisissez de l'utiliser. Le contrôle : « Quels outils du serveur tailspin vois-tu ? » ; il doit citer `search_games` et `get_game`.
- **Étape 3** : Ctrl+O fait défiler les vues ; la vue *Verbose* montre chaque appel d'outil.
- **Étape 4** : Ctrl+N, sur le même dossier, au lieu de `/exit` et de la relance.

Étape 1 : faites-le écrire (14 min)

Sur une branche neuve (`git switch -c mcp-catalogue`) : `claude --model sonnet`, puis :

```
Écris un serveur MCP en stdio pour le catalogue, dans
mcp/catalog-server.ts, avec le SDK officiel v2
(@modelcontextprotocol/server, et zod). Deux outils en lecture
seule, qui réutilisent src/lib/ : search_games (un texte, cherché
dans le titre, la catégorie et l'éditeur) et get_game (par id).
La logique va dans src/lib/, testée par Vitest ; le serveur ne
fait que l'exposer. Ne lance pas d'agent.
```

Puis **lisez le serveur** : la description de chaque outil (c'est elle qui décide quand Claude l'appelle), et où partent ses messages d'erreur (stdout porte le protocole : un `console.log` casse le serveur). Fini en avance ? Lisez les tests : que prouvent-ils, et que ne prouvent-ils pas ?

Étape 2 : branchez-le (7 min)

Dans Claude, `/exit` ; puis, dans le terminal :

```
claude mcp add --scope project tailspin -- npx tsx mcp/catalog-server.ts
claude --model sonnet
```

- Au lancement, « New MCP server found in this project » : le curseur est sur « Continue without… » ; choisissez **Use this MCP server**.
- Dans la session, `/mcp` : **✔ tailspin · 2 tools** ; puis *View tools* : `search_games` et `get_game`, *read-only*.

Fini en avance ? `/context` : la ligne « MCP tools ».

Étape 3 : servez-vous-en (9 min)

1. Demandez : « Quel est le jeu le mieux noté de chaque catégorie ? »
2. **Ctrl+O : par où est-il passé ?** Un appel à `tailspin`, ou autre chose ?
3. Puis : « Même question, en passant uniquement par les outils du serveur tailspin. »
4. Et hors catalogue : « Combien a levé Repo Rulers ? »

Fini en avance ? Une demande vague : « Un jeu pour ce soir ? » Que fait-il, sans consigne ?

Question	Votre réponse
À la première question, par où est-il passé ?	
Hors catalogue : a-t-il dit qu'il ne savait pas, ou avancé un chiffre ?	

Étape 4 : confiez-le à un sous-agent (15 min)

```
Crée le sous-agent concierge, dans .claude/agents/concierge.md :
il répond aux questions sur le catalogue en suivant les règles de
backlog/09-backer-concierge.md, sur Haiku, avec pour seuls outils
ceux du serveur tailspin. Déclare ce serveur dans son en-tête, en
ligne, avec la même commande que dans .mcp.json, puis retire-le
de .mcp.json : la conversation principale ne doit plus le voir.
Ne lance pas d'agent.
```

1. Relisez son en-tête : `model`, `tools`, `mcpServers`.
2. `/exit`, puis `claude --model sonnet` : un dossier `agents` neuf se charge au démarrage.
3. « Demande au concierge : quel jeu de stratégie est le mieux noté, et combien a-t-il levé ? »

Fini en avance ? `/context` : plus d'outil tailspin dans la conversation. Puis une demande vague au concierge : il doit poser une seule question.

Si ça coince

- Claude prend l'ancien SDK, `@modelcontextprotocol/sdk` : ça marche aussi, seuls les imports changent ; ne recommencez pas.
- `tailspin.db` absent : `npm run db:setup` dans le checkout principal.
- Le serveur lancé à la main ne rend pas la main : c'est normal, il attend un client sur stdin. Ctrl+C.
- « Pending approval », ou « Continue without » choisi par réflexe : `claude mcp reset-project-choices`, puis relancez `claude` et choisissez « Use this MCP server ».
- « ✘ Failed to connect » : lancez `claude` à la racine (le chemin du serveur est relatif) ; lancez `npx tsx mcp/catalog-server.ts` à la main pour voir l'erreur ; puis `/mcp`, Reconnect.
- Windows, « Connection closed » : `claude mcp remove tailspin -s project`, puis le même `claude mcp add` avec `cmd /c` avant `npx`.
- Windows, le concierge part sans outils alors que l'étape 2 a demandé `cmd /c` : son en-tête doit le reprendre ; demandez-le à Claude.
- App desktop, le serveur ou le concierge ne se charge pas : une nouvelle session (Ctrl+N), *Local*, sur la racine du projet, sans worktree. Une session en worktree lit `.mcp.json` et `.claude/agents/` à la racine du checkout principal ; une session *Cloud* ne lance pas de serveur local.
- Claude n'appelle pas le serveur : c'est la leçon de l'étape 3, il a un chemin plus court ; « en passant uniquement par les outils du serveur tailspin ».
- Hors du mode Auto, Claude demande l'accord pour chaque outil MCP : `mcp__tailspin__*` dans `permissions.allow` lève la question pour ce serveur.
- « 'concierge' would be spawned with zero tools — refusing » : son serveur ne s'est pas chargé. La session n'a pas été relancée après la création de `.claude/agents/`, ou le dossier n'est pas de confiance (`claude` une fois, acceptez).
- Le concierge n'est pas appelé : nommez-le, « Demande au concierge ».

- La session principale voit encore les outils tailspin : `.mcp.json` pas vidé, ou session pas relancée.

Question	Votre réponse
Dans ma mission, l'outil que je brancherais en premier	
Une CLI que Claude connaît déjà, un serveur publié par l'éditeur, ou le mien ? Qui doit l'autoriser chez le client ?	

Plan d'adoption à 30 jours

Un plan pour votre équipe, construit ligne par ligne : une ligne à la fin de chaque demi-journée, puis la consolidation en clôture du J2. Si la politique IA de votre entreprise est en cours de définition, ce plan peut la nourrir.

Ligne 1 · fin du J1 matin : le contexte

	Votre réponse
Dès lundi, dans le CLAUDE.md de mon équipe, je change	
Je propose d'ajouter la règle	

Ligne 2 · fin du J1 après-midi : hooks et skills

	Votre réponse
Le hook que je mets en place dans mon équipe	
Le skill que je partage	

Ligne 3 · fin du J2 matin : composer et piloter

	Votre réponse
Ce que je mets en place dans mon équipe pour composer et piloter	

Exemples : *« une chaîne interview → spec → tickets sur notre prochain ticket »* ; *« une condition `/goal` écrite pour chaque tâche longue »*. D'autres pistes : le skill « à créer » de votre plan de composition ; la grille « quand vous rentrez dans la boucle » en revue de PR ; garder les specs dans le repo, à côté du code ; « toute fonctionnalité de plus d'une demi-journée passe par une spec écrite », une ligne de politique IA concrète.

Ligne 4 · fin du J2 après-midi : paralléliser

	Votre réponse
La correction de harness que je fais lundi (celle que j'ai entourée dans la grille des tentations)	
Ma première parallélisation, à deux sessions	

Consolidation · en clôture du J2

Regroupez vos lignes 1 à 4 en un plan de quatre semaines. Un ticket de votre mission par ligne : aujourd'hui, vous avez travaillé sur Tailspin ou sur votre projet ; lundi, c'est chez le client. Notez le ticket ou le sujet réel où la pratique s'applique en premier. Un résultat mesurable par ligne : pas « utiliser les skills », mais « le skill de revue se déclenche sans être nommé sur nos trois prochaines PR ». Un responsable par ligne : une ligne sans nom n'est faite par personne. Un plan qui tient en quatre lignes est un plan qu'on fait. Les exemples en italique donnent le niveau de précision attendu.

Semaine	Ce que je mets en place	Sur quel ticket de ma mission	Comment je sais que ça marche	Qui
Semaine 1	*Exemple : le CLAUDE.md et la règle du J1, commités, relus par l'équipe*	*Exemple : le repo de la mission*	*Exemple : la PR est relue et mergée*	
Semaine 2	*Exemple : un hook et un skill partagés*	*Exemple : le prochain ticket qui touche une migration*	*Exemple : le skill se déclenche sans être nommé*	
Semaines 3–4	*Exemple : une chaîne interview → spec → tickets, et une première parallélisation à deux sessions*	*Exemple : le prochain ticket d'une demi-journée*	*Exemple : le temps de relecture par PR*	

Pour la colonne « Comment je sais que ça marche », voyez la slide « Mesurer sans se tromper » ; le détail est dans le récap (« Mesurer les résultats, pas l'activité »). Relisez la checklist context engineering : elle montre ce que vous avez oublié dans votre plan.

Checklist context engineering

À cocher dans les semaines qui viennent. C'est le livrable 5 du programme ; relisez-la en écrivant votre plan d'adoption : elle montre ce que vous avez oublié.

Partie J1 : le contexte, le CLAUDE.md, les hooks et les skills

- [ ] Je commence toute première session sur un repo en Plan mode.
- [ ] Je tape `/context` avant une grosse tâche, et quand Claude semble oublier une consigne.
- [ ] J'ai réglé l'auto-compaction vers 70 % de la fenêtre (`/autocompact 700k` sur une fenêtre de 1M).
- [ ] Je fais `/clear` entre deux tâches sans rapport.
- [ ] Je compacte avec une consigne (`/compact garde …`), pas à l'aveugle.
- [ ] Quand Claude part dans une mauvaise direction, je reviens en arrière (Esc Esc) au lieu de corriger par-dessus.
- [ ] Mes prompts référencent les fichiers (`@chemin`), montrent un modèle, posent les limites et donnent un critère de vérification.
- [ ] Pas de tests sur la zone ? Je demande d'abord le test.
- [ ] Mon CLAUDE.md fait moins de 200 lignes et ne contient que ce que Claude ne peut pas deviner.
- [ ] Il désigne des fichiers modèles (golden files) au lieu d'énumérer des règles de style.
- [ ] Les consignes propres à une partie du code sont dans `.claude/rules/`, avec `paths`.
- [ ] Une erreur qui revient devient une règle ; une procédure qui revient devient un skill.
- [ ] Ce qui doit arriver à chaque fois (formatage, protection des secrets, tests) est dans un hook, pas dans le CLAUDE.md.
- [ ] Mes secrets sont protégés par une règle `deny` et un hook `PreToolUse`, et je sais ce qu'ils ne couvrent pas : le sandbox, lui, bloque les commandes shell quel que soit leur texte.
- [ ] Je vérifie dans `/context` le poids de mes serveurs MCP et de mes skills, et je débranche ce qui ne sert pas.

Partie J2 : workflows, loops, parallélisation

- [ ] Je dose : si le diff tient en une phrase, pas de plan ; le Plan mode quand l'approche est incertaine ; l'interview et la spec pour un ticket d'une demi-journée et plus.
- [ ] Pour une grosse fonctionnalité, je me fais interviewer, la spec va dans un fichier, et je l'exécute dans une nouvelle session.
- [ ] À la fin de l'interview, je tape `/to-spec` au lieu de répondre « oui, vas-y ».
- [ ] Mes tickets sont des tranches verticales, vérifiables seules, avec leur ligne « Blocked by » ; jamais un découpage par couche.
- [ ] Mon workflow laisse des fichiers courts (spec, tickets, plan) ; chaque implémentation repart de ces fichiers, en contexte frais.
- [ ] Je compose mes skills plutôt que d'adopter une méthode en bloc ; j'emprunte aux méthodes leurs bonnes idées.
- [ ] Je choisis mon niveau de supervision tâche par tâche : in the loop pour la conception et ce qui est risqué, on the loop pour un ticket bien spécifié et testé.
- [ ] Quand j'interviens, je corrige la cause (le harness, la spec ou le code) plutôt que la sortie.

- [ ] Mes conditions `/goal` ont un état final mesurable, une vérification énoncée, les contraintes qui comptent et un plafond de tours, tous visibles dans la sortie de Claude.
- [ ] Une loop qui corrige du code n'a pas le droit de modifier les tests qui la jugent.
- [ ] Une loop sans surveillance tourne sur une branche, en mode Auto, après un run pilote sur un petit périmètre ; je surveille les tokens dans le statut de `/goal` (et, en non interactif, je plafonne la dépense avec `--max-budget-usd`), et je fixe toujours `--max-iterations` avec `ralph-loop`.
- [ ] Un worktree par agent qui écrit du code : jamais deux sessions dans le même dossier.
- [ ] Mes worktrees sont prêts à l'emploi : `.claude/worktrees/` dans le `.gitignore`, `.worktreeinclude` pour les fichiers ignorés, un script d'amorçage versionné pour les dépendances.
- [ ] Je délègue à un sous-agent ce qui produit une sortie volumineuse (recherche, logs, tests), avec un brief : objectif, format de sortie, consignes sur les outils et les sources, limites.
- [ ] Le rapport d'un sous-agent est une hypothèse : je vérifie ses citations avant d'agir.
- [ ] Le brief d'un sous-agent lui interdit de lancer des agents.
- [ ] Avant de brancher un serveur MCP, je vérifie qu'une CLI et un skill ne suffisent pas ; je l'approuve comme une dépendance, et je le confie à un sous-agent quand une interface étroite suffit.
- [ ] Je choisis le modèle selon la phase : Opus pour concevoir et relire, Sonnet pour implémenter.
- [ ] Le graphe des tickets décide de ce qui part en parallèle ; je commence à deux sessions, et j'en ajoute seulement si ma relecture et mon attention suivent.
- [ ] Je mesure des résultats (temps de relecture par PR, changements mergés du premier coup, retours sur la spec), pas l'activité.

Auto-évaluation : les 6 objectifs

En clôture du J2 : pour chaque objectif du programme, cochez où vous en êtes. **1** = je ne saurais pas le refaire seul ; **2** = je le refais avec le livret ouvert ; **3** = je le refais seul, sans le livret ; **4** = je peux le montrer à mon équipe.

Objectif du programme	1	2	3	4	Où le livret le reprend
1. Piloter la fenêtre de contexte	☐	☐	☐	☐	J1 matin — Récap : « Piloter la fenêtre de contexte », « La règle des 70 % », « Les 4 modes de défaillance »
2. Rédiger des prompts précis	☐	☐	☐	☐	J1 matin — Récap : « Un prompt précis : quatre verbes » ; J1 matin — Cahier : « Exercice express : réécrire un prompt »
3. Configurer son harness : CLAUDE.md, règles, hooks	☐	☐	☐	☐	J1 matin : « CLAUDE.md, le README des agents » et l'Atelier 1 ; J1 après-midi : « Les hooks : du conseil à la garantie » et l'exercice `.env`
4. Concevoir des skills composables	☐	☐	☐	☐	J1 après-midi : « Les skills : du savoir ou des procédures, à la demande », « Partager un skill » et « Exercice : créer votre skill » ; J2 matin — Récap : « La chaîne de Matt Pocock »
5. Composer des workflows spec-driven et choisir son niveau de supervision	☐	☐	☐	☐	J2 matin — Récap : la spec, la chaîne, doser, superviser, les loops ; J2 matin — Cahier : l'Atelier 2, le plan de composition, « `/goal` à blanc », la condition d'arrêt
6. Paralléliser avec worktrees et sous-agents	☐	☐	☐	☐	J2 après-midi — Récap : worktrees, sous-agents, le prompt du pipeline ; J2 après-midi — Cahier : la préparation des worktrees, l'Atelier 3 et ses deux prompts, le sous-agent concierge de l'exercice MCP

Un objectif à 1 ou 2 : reprenez la partie indiquée. Pour le 6, refaites l'Atelier 3 au calme, sur un ticket de la semaine suivante, avec le prompt du pipeline.